

Warehouse and Inventory Management using Reinforcement Learning

*Project report submitted to
Visvesvaraya National Institute of Technology, Nagpur in partial fulfillment of the
requirements for the award of the degree*

Bachelor of Technology in Computer Science and Engineering

by

Izaan Sheikh
(BT21CSE033)

Avirat Shambharkar
(BT21CSE040)

Ritesh Bansod
(BT21CSE119)

under the guidance of

Dr. Ravindra B. Keskar



Department of Computer Science and Engineering

Visvesvaraya National Institute of Technology

Nagpur 440010 (India)

2021 – 2025

Warehouse and Inventory Management using Reinforcement Learning

*Project report submitted to
Visvesvaraya National Institute of Technology, Nagpur in partial fulfillment of the
requirements for the award of the degree*

Bachelor of Technology in Computer Science and Engineering

by

Izaan Sheikh
(BT21CSE033)

Avirat Shambharkar
(BT21CSE040)

Ritesh Bansod
(BT21CSE119)

under the guidance of

Dr. Ravindra B. Keskar



Department of Computer Science and Engineering

Visvesvaraya National Institute of Technology

Nagpur 440010 (India)

2021 – 2025

Department of Computer Science and Engineering
Visvesvaraya National Institute of Technology, Nagpur



Declaration

We, **Izaan Shaikh (BT21CSE033)**, **Avirat Shambharkar (BT21CSE040)**, **Ritesh Bansod (BT21CSE119)**, hereby declare that this project work titled “**Warehouse and Inventory Management using Reinforcement Learning**” is carried out by us in the Department of Computer Science and Engineering of Visvesvaraya National Institute of Technology, Nagpur. The work is original and has not been submitted earlier whole or in part for the award of any degree/diploma at this or any other Institution / University.

Date:

Sr. No.	Name	Enrollment No.	Signature
1.	Izaan Shaikh	BT21CSE033	
2.	Avirat Shambharkar	BT21CSE040	
3.	Ritesh Bansod	BT21CSE0119	

Certificate

This to certify that the project titled “**Inventory and Warehouse Management using Reinforcement Learning**”, submitted by **Izaan Shaikh, Avirat Shambharkar, Ritesh Bansod** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**, VNIT Nagpur. The work is comprehensive, complete and fit for final evaluation.

Head, Dept of
Computer Science and Engineering,
VNIT, Nagpur

Date:

Dr. R. B. Keskar
Associate Professor, Dept of
Computer Science and Engineering,
VNIT, Nagpur

Acknowledgement

We would like to extend our gratitude to our project guide Dr. R B Keskar, Associate Professor, Department of Computer Science and Engineering, VNIT Nagpur, for his invaluable guidance and support throughout the course of this project. Our sincere thanks to him for his time to solve our doubts, and help to improvise our work at each step.

We also thank our panel members, Dr. D. D. Shrimankar, Assistant Professor and Dr. S. R. Sathe, Professor, Department of Computer Science and Engineering, VNIT Nagpur, for offering insights that allowed us to expand our project.

Thank you.

Izaan Jakir Sheikh

Avirat Shambharkar

Ritesh Bansod

Abstract

Businesses operate far and wide the map of India. These are seen gone through producer or manufacturer to the final customer. This process of product manufacturing to consumption through the logistics and supplier coordination is known as Supply Chain. Supply Chain combined with computer science can help boost the business operations effectively and efficiently. Indian economy can be boosted heavily if the supply chain optimization is initiated. Many startups have come into being and are also doing great work in this optimization process.

Supply chain management consists of various phases right from factory to the customer the product is manufactured and delivered to the proper hands. The supply chain has various problems like inventory management i.e. on demand forecasting, warehouse management i.e. on supplier coordination for transportation logistics. Inventory management and warehouse management, as the name tells and suggests us that we manage the inventory and warehouse items. We check the inventory levels, order count and back orders. Similarly, the warehouses across the map are mapped accordingly and hence managed using this warehouse management service.

The machine learning branch of computer science also plays a pivotal role in improving the different aspects of the supply chain if done right. We found that reinforcement learning a branch of machine learning can be extensively used in this domain to help improve the efficiency of the domains discussed previously.

Commented [RK1]: A bit more about work done and results obtained

List of Figures

Fig 3.1 (a) Raw data of DateItems	19
Fig 3.1 (b) Raw Metadata of Warehouse	19
Fig 3.4 (c) Format date	21
Fig 4.2 (a) Frontend display when first loaded	25
Fig 4.2 (b) Frontend display when adding a new warehouse	26
Fig 4.2 (c) Inventory display	27
Fig 4.2 (d) Map Visualization	28
Fig 4.2 (e) Forecasting Interface	28
Fig 4.4 (a) DQN model Architecture	31
Fig 4.4 (b) DQN model Architecture code	33
Fig 4.4 (c) PPO model Architecture	35
Fig 4.4 (d) PPO model Architecture code	37
Fig 4.5 (a) User Database structure	39
Fig 4.5 (b) Warehouse Database structure	40
Fig 5.1 (a) Prediction result DQN	42
Fig 5.2 (a) Best warehouse PPO	43
Fig 5.3 (a) All days Item List after making prediction	45
Fig 5.3 (b) PPO all nearby warehouse were item available	46
Fig 5.3 (c) Inventory of mobile and tablet graph	47

List of Tables

Table 2.3 (a) Reinforcement Learning Techniques	16
Table 2.3 (b) Comparison of Techniques	16
Table 4.1 (a) Libraries and Frameworks Used	23
Table 4.5 (a) Users Collection Schema	38
Table 4.5 (b) Warehouse Collection Schema	39
Table 5.4 (a) DQN and PPO comparison	48

Table of Contents

INTRODUCTION.....	1
1.1 Supply Chain Management.....	1
1.1.1 Background	1
1.1.2 Problem Statement	3
1.1.3 Solution	4
1.1.4 Outcomes.....	5
1.2 Contribution.....	5
1.3 Organization of Thesis.....	6
LITERATURE SURVEY	8
2.1 Traditional Inventory Management Techniques	8
2.1.1 Economic Order Quantity (EOQ)	8
2.1.2 Just-In-Time (JIT).....	9
2.1.3 Reorder Point (ROP) and Safety Stock Models.....	9
2.1.4 Limitations of Traditional Techniques.....	10
2.2 Machine Learning Management Techniques	10
2.2.1 Regression and Time Series Model	10
2.2.2 Deep Learning Models.....	11
2.2.3 Advantages and Limitations	12
2.3 Reinforcement Learning in Warehouse Management	12
2.3.1 Deep Q-Networks (DQN) for Inventory Forecasting.....	13

2.3.2 Proximal Policy Optimization (PPO) for Restocking Location Selection	14
DATA PREPROCESSING	18
3.1 Raw Data Overview	18
3.1.1 Inventory Data (dateItems)	18
3.1.2 Warehouse Metadata	19
3.2 Inventory Forecasting (DQN) Preprocessing	20
3.2.1 Flattening Nested Records	20
3.3 Restock Location Selection (PPO) Preprocessing	20
3.3.1 Input Format	21
3.4 Handling Missing and Corrupt Data	21
DESIGN AND IMPLEMENTATION	22
4.1 System Components	22
4.1.1 Libraries	23
4.2 Frontend Architecture	25
4.3 Backend Architecture	29
4.4 Machine Learning Models	29
4.4.1 DQN for Inventory Forecasting	29
4.4.2 PPO for Warehouse Location Selection	34
4.5 Database Architecture	38
4.5.1 User Collection	38
4.5.2 Warehouse Collection	39

4.6 Integration with the Backend and Frontend.....	40
RESULTS AND ANALYSIS	41
5.1 Inventory Forecasting with DQN.....	41
5.1.1 Model Overview	41
5.1.2 Prediction Visualization	42
5.2 Warehouse Location Selection with PPO	43
5.2.1 Model Overview	43
5.2.2 Location Visualization	43
5.3 Frontend Integration Results.....	44
5.3.1 Inventory Dashboard.....	44
5.3.2 Warehouse Location Map	46
5.4 Comparative Analysis	48
5.5 Summary	48
CONCLUSION AND FUTURE WORK.....	49
REFERENCES	51

CHAPTER 1

INTRODUCTION

Supply chain consists of many parts. We can see that a product starts its journey right from the factory to the retailer and then to the customer goes through many points like wholesaler, distributor and then retailer. Warehouse management **plays** a key role in supply chain that **helps** in timely restocking, optimized storages **are optimized**, and efficient usage of resource. In today's **world**, the traditional manual system fails to address these needs because of non-adaptability and non-predictiveness. With the development of machine learning and reinforcement learning techniques we **are** able to develop intelligent warehouse systems.

This is a project about a Warehouse Management System (WMS) using **Deep Q-Network (DQN)** for **inventory predication** and **Proximal Policy Optimization (PPO)** for **restocking based on warehouse location**. The aim is to develop a full stack application that not only tracks daily sales and inventory but also suggests smartly how much to restock and from which nearby warehouse. The website is developed on a full stack architecture (Node.js backend, MongoDB, Python ML, and HTML/JS frontend).

1.1 Supply Chain Management

1.1.1 Background

The supply chain is the lifeline of modern commerce, that covers the complete process of producing and delivering goods from raw material sourcing to the final delivery to the consumers. Efficient supply chain management (SCM) is important to minimize costs of operations, improving customer satisfaction, and gaining a competitive advantage. Traditional SCM has relied on deterministic models, statistical forecasting, and rule-based systems to handle logistics, demand planning, and inventory management.

However, today's real-world supply chains are increasingly dynamic and complex. Unpredictable shifts in customer demand, supplier delays, transportation disruptions, and geopolitical tensions challenge the robustness of traditional methods. As highlighted by Swaroop Reddy Gayam et al. [1], artificial intelligence (AI) techniques, particularly predictive maintenance, demand forecasting, and inventory optimization, are reshaping supply chain operations by enabling systems to predict disruptions and adapt proactively. The integration of AI not only enhances supply chain visibility but also enables dynamic decision-making, where models continuously learn from operational data to optimize inventory levels, minimize downtime, and align supply with fluctuating demand.

Recent, advancement in **reinforcement learning** have made it possible for developing agents that can learn optimal policies from data. **Deep Q-Networks (DQN)** is used for environments with discrete action spaces like determining daily restock quantities, while **Proximal Policy Optimization (PPO)** is often used for tasks like selecting the best warehouse location by considering multiple factors such as how far it is distance, whether it is available, and time.

Why is warehouse inventory management a challenging task?

- **Dynamic Sales Behaviour:** Customer demand varies daily and by season, making it difficult to maintain optimal inventory without the prediction on the inventory.
- **Space Constraints:** Warehouses have limited storage space, requiring careful allocation and restocking decisions per item.
- **Multiple Supply Sources:** Selecting the best restocking warehouse involves taking in consideration stock availability, delivery distance, and travel time, etc.
- **Limitations of models:** Traditional models struggle to handle the complex relationships between sales, inventory, and the patterns among them, machine learning-based methods are helpful in these cases.

1.1.2 Problem Statement

Traditional inventory systems lack intelligent prediction capabilities, often resulting in overstocking or stockouts. Hence the problem identified **is** to efficiently handle the inventory management tasks using reinforcement learning. Our goal is to predict the next day's inventory requirement for each item using past sales and inventory data, and also to suggest the best nearby supplier warehouse for restocking.

The problem can be framed into two parts:

- Predicting inventory levels for each item in every warehouse using Deep Q-Learning (DQN).
- Restocking Location Selection using PPO based on item availability, distance, and travel duration.

Why Reinforcement Learning (RL)?

RL, an advanced machine learning method, that gives adaptive decision-making and helps optimize SCM processes by responding dynamically to changing conditions, learning from past outcomes, and continuously improving decisions.

As outlined by Sutton et al. [2], RL frameworks excel when an agent must make a series of interdependent decisions under uncertainty, especially when the underlying system dynamics are unknown or too complex to model explicitly. Unlike traditional optimization techniques that rely on static datasets or predefined rules, RL agents develop strategies by exploring and exploiting the environment, gradually improving their policies through trial-and-error experiences. This ability to continuously adapt based on reward feedback makes RL particularly effective for real-world supply chains, where factors such as demand patterns, supplier reliability, and transportation logistics frequently fluctuate in unpredictable ways.

It is best suited for sequential, feedback-driven environments like inventory management and supplier selection, where every action impacts future system states.

Unlike traditional models, RL does not rely on predefined rules or static data sets, which make it ideal for managing the uncertainty and supply chain fluctuations.

1.1.3 Solution

The proposed solution uses reinforcement learning techniques for autonomous decision-making in warehouse environments:

Inventory Prediction using DQN: Deep Q-Learning is a powerful technique in Reinforcement Learning (RL) that combines traditional Q-Learning with deep neural networks to manage complex environments with large or continuous state spaces. For each item, we train a DQN agent using past inventory and sales data. The agent learns how much to restock daily by optimizing rewards based on stock sufficiency and space utilization.

Supplier Warehouse Selection using PPO: Proximal Policy Optimization (PPO) is a policy gradient algorithm in reinforcement learning developed by OpenAI. It is used to train agents safely and efficiently, especially in complex environments with high-dimensional state and action spaces. For selecting the best warehouse to restock from, we use the model so that it learns from features like stock availability at nearby warehouses, distance, and delivery duration.

Both models are deployed through a full stack system:

- **Frontend (HTML/CSS/JS):** Displays inventory data and lets users train and make predictions.
- **Backend (Node.js + Express):** Handles routing and communication with MongoDB and Python ML scripts.
- **Machine Learning Layer (Python):** Trains and predicts using DQN and PPO models, storing results in MongoDB.

1.1.4 Outcomes

The smart warehouse management system enables the following outcomes:

- **Optimized Inventory Control:** Predicts required inventory per item, reducing both stockouts and overstock scenarios.
- **Improved Restocking Decisions:** Selects the best restocking warehouse from where to restock.
- **Increased Operational Efficiency:** Automation helps things to run smoothly and reduces the errors.
- **Data-Driven Insights:** Helps warehouse managers analyses the demand trends and optimize item storage allocation.
- **Scalable Design:** Easily extendable so that we can add more warehouses, new items, or different product.

1.2 Contribution

Commented [RK2]: Give pointwise list of contributions

The project was executed in two major phases.

In **Phase 1**, the initial focus was on foundational research and base model development. During, **Evaluation 1** comprehensive literature review was done to explore the applications of Reinforcement Learning (RL) in Supply Chain Management (SCM), with the focus on identifying the most suitable techniques for solving inventory and warehouse management problems. In **Evaluation 2**, the Q-learning algorithm was implemented to simulate and optimize warehouse operations by learning replenishment policies based on environmental interactions. In **Evaluation 3**, where the Deep Q-Network (DQN) technique was integrated to improve the learning performance in high-dimensional state spaces, and enhance the agent's ability to make more accurate restocking decisions.

In **Phase 2**, the focus was on more advanced implementation and integration and completing the project. **Evaluation 4** involved the development of a Proximal Policy Optimization (PPO) model specific to warehouse and supplier selection tasks. Alongside this, a web-based interface was built to visualize and interact with the reinforcement

learning models. Finally, in **Evaluation 5**, both the DQN and PPO models were successfully integrated into the website, provide simulation and interaction for warehouse management. This phase also included code debugging and performance testing to ensure system reliability and accuracy.

1.3 Organization of Thesis

This thesis is design to systematically address the problem of supply chain optimization through reinforcement learning, particularly focusing on inventory management and supplier selection in a multi-warehouse environment.

Chapter 1 introduces the motivation behind optimizing supply chains, especially within the context of Indian logistics. It outlines the key challenges in inventory and warehouse management and discusses how reinforcement learning methods such as Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO) can provide data-driven, adaptive solutions This chapter explains the main problem, outlines what the project covers, and talks about what results we hope to see and how these techniques can be useful in real life.

The remaining of the thesis is organized as follows:

- **Chapter 2** provides a detailed description of the dataset used, including its sources, structure, and attributes It includes a broad review of the existing research, covering traditional forecasting techniques, machine learning methods, and how reinforcement learning is being used in supply chain management.
- **Chapter 3** goes over how to prepare the data and create the right features before training the model. This involve cleaning up the data, putting it into a consistent format, and setting up how the system learns from each step by linking states, actions, and rewards so it can understand and improve over time.
- **Chapter 4** presents the implementation of the proposed models It explains how the DQN is built and trained to predict inventory levels, and how the PPO model is used to select the warehouse location. The chapter also covers how the environment design, choices made for hyperparameters, and learning policies.

- **Chapter 5** discusses the experimental results, including performance metrics, comparisons with baseline models, and an analysis of model behaviour across different warehouse and item scenarios. Observations are supported up by visualizations and an understanding of how the learned policies work.
- **Chapter 6** gives the overall conclusion we can draw from these results and future work to be done in this field.
- **Chapter 7** contains the references and citations to all research papers, articles, and sources consulted during the course of the study.

Together, these chapters aim to provide a comprehensive framework for understanding and applying deep reinforcement learning to supply chain optimization. The next chapter presents the dataset and literature survey.

CHAPTER 2

In this chapter we will discuss the existing techniques, and research studies related to warehouse management and inventory forecasting. We also discuss the role of reinforcement learning in optimizing warehouse operations, including inventory prediction and supplier selection.

LITERATURE SURVEY

2.1 Traditional Inventory Management Techniques

Traditional techniques of inventory handling have been the backbone of warehouse management for decades. These techniques mainly focus on maintaining the optimal stock levels, minimizing holding and shortage costs, and ensuring product availability. As much as they are used across the globe, these approaches mainly operate on static rules and assumptions that make it difficult for them to adapt to the dynamic nature of supply chain environments.

2.1.1 Economic Order Quantity (EOQ)

The Economic Order Quantity (EOQ) model calculates the optimal order quantity that minimizes the total inventory management cost, balancing ordering and holding costs. It assumes that demand remains constant, lead times are stable, and holding costs do not vary over time. While EOQ offers a mathematically sound approach under idealized conditions, it often struggles in real-world scenarios characterized by fluctuating demand, multiple warehouses, inflationary pressures, and deteriorating goods. Recent studies, such as the work by Mallick et al. [3], have shown that the classical EOQ model requires significant adaptations to remain relevant in dynamic environments. Their research proposed a modified EOQ model that accounts for stock-dependent demand, inflation effects, and the deterioration of inventory over time. This highlights the limitations of the traditional EOQ model and reinforces the need for more adaptive and intelligent inventory

management strategies, such as those based on machine learning and reinforcement learning, especially in volatile supply chain networks.

2.1.2 Just-In-Time (JIT)

Just-in-Time (JIT) is a demand-driven inventory strategy where materials and products are replenished only as needed in the production process, minimizing holding costs to the bare minimum. Originally developed in Japan, particularly in the automobile sector by companies like Toyota, JIT focuses on reducing inventory waste, optimizing workflows, and enhancing operational efficiency. As discussed by Cheng and Podolsky [4], the JIT philosophy extends beyond mere inventory control, influencing production scheduling, supplier management, and overall supply chain coordination. While JIT significantly lowers storage costs and streamlines operations, it is heavily dependent on the reliability and punctuality of suppliers. Any disruption, delay, or inconsistency in the supply chain can severely impact production continuity, making JIT systems vulnerable in volatile or unpredictable environments. Therefore, while JIT offers substantial efficiency gains under stable conditions, it poses considerable risks if not supported by resilient and synchronized supply chain networks.

2.1.3 Reorder Point (ROP) and Safety Stock Models

The Reorder Point (ROP) model automatically triggers restocking when inventory falls below a predetermined threshold, ensuring that stock levels are maintained to meet customer demand. Safety stock is simultaneously maintained as a buffer against fluctuations in demand or lead time variability. As discussed by Teuku Faisal Umry et al. [5], ROP models are crucial in environments with random demand variations, particularly under a continuous review policy, which monitors inventory levels in real-time to avoid stockouts. However, despite their operational simplicity, traditional ROP models are non-predictive and can lead to either stockouts or overstocking if not recalibrated frequently. Static ROP thresholds, when disconnected from real-time demand trends and lead time uncertainties, may result in significant financial inefficiencies such as excessive working capital being tied up or service level failures. This highlights the need for more dynamic and intelligent inventory management approaches, particularly in volatile supply chains.

2.1.4 Limitations of Traditional Techniques

These models are normally based on static parameter assumption, such as the application of fixed rates of demand and some deterministic lead times. These models suffer from incorporating today's real time data, high market fluctuation, or between Warehouse relations. Traditional approaches cannot learn from historical data or dynamically change strategies, which is essential for managing modern supply chains.

2.2 Machine Learning Management Techniques

As data availability and computational capabilities increased, machine learning (ML) has become a valuable aid to enhance forecasting precision and operational-level decision-making for inventory management. ML-based inventory forecasting techniques offer the ability to see the complex, nonlinear relationships in data and they outperforming traditional methods in dynamic environments.

2.2.1 Regression and Time Series Model

Regression and time series models have long served as foundational techniques for forecasting inventory demand. Supervised learning algorithms such as Linear Regression, Decision Trees, Random Forests, and Gradient Boosting Machines (GBMs) are commonly employed to predict future inventory requirements by utilizing historical sales data, seasonal patterns, and promotional activity as input features. Random Forests, an ensemble-based learning algorithm, aggregate predictions from multiple decision trees, thereby reducing the risk of overfitting and improving the model's generalization capabilities. Gradient Boosting Machines, on the other hand, sequentially build an ensemble of weak learners, with each tree correcting the errors made by the previous one, thereby creating a highly accurate predictive model.

As highlighted by Akbar Abbaspour Ghadim Bonab [6], machine learning models like Random Forest and Gradient Boosting have demonstrated superior performance over traditional statistical models such as ARIMA, particularly when dealing with non-linear relationships and complex demand patterns in supply chain environments. The study emphasizes that while time series approaches provide useful trend analysis, machine

learning models offer higher predictive accuracy and adaptability by leveraging rich historical data and feature interactions. This advancement sets the stage for more dynamic inventory forecasting systems, especially critical in volatile and high-variability supply chains.

2.2.2 Deep Learning Models

Deep learning methods such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and Temporal Convolutional Networks (TCNs) have shown significant promise in remembering long-term dependencies and sequential patterns in demand forecasting tasks. Unlike traditional models, deep learning architectures can automatically learn complex feature interactions and seasonality from raw time series data, making them highly effective for inventory and sales prediction.

Recurrent Neural Networks (RNNs) are a class of neural networks that are particularly suited for sequential data, where the prediction at a given time depends on previous observations. RNNs introduce internal memory through recurrent connections, allowing them to model temporal dependencies. However, standard RNNs struggle with capturing long-term dependencies due to issues like vanishing gradients.

To overcome these limitations, Long Short-Term Memory (LSTM) networks were developed. LSTMs are a specialized form of RNNs designed to remember information over extended sequences. As discussed by Cristian Cracan [7], LSTM architectures are particularly effective in capturing complex temporal dynamics in retail sales and demand forecasting tasks. Cracan demonstrated that LSTM models outperform traditional statistical methods and even simple RNNs when dealing with real-world sales datasets characterized by seasonality, noise, and irregular demand patterns. The ability of LSTMs to selectively forget, remember, and update information across long sequences makes them highly suitable for future inventory forecasting in supply chain systems. However, despite their effectiveness, LSTMs can become computationally expensive and require extensive tuning of hyperparameters, and they may struggle with extremely long sequences or very large datasets where models like Temporal Convolutional Networks (TCNs) or Transformers might perform better.

2.2.3 Advantages and Limitations

Machine learning models provide adaptability, higher accuracy, and the ability to process a large number of variables these are the advantages and they also require a well-organized and properly labeled data and limitations are its decision-making ability and it is used for prediction rather than optimization.

Although the machine learning models offer improvements over traditional methods, they still face challenges in automating decision-making and being seamlessly integrated with warehouse operations.

2.3 Reinforcement Learning (RL) in Warehouse Management

Commented [RK3]: Basics of RL and Q-learning can be mentioned in detail.

The agent takes the actions in the world and then it receives the feedback in the form of rewards or penalties based upon the outcomes of those actions. Then the feedback is used to modify the decision-making process of the agent so that it can improve and learn over time. In supervised learning, we have seen that the models are being trained over the labeled data, In RL models learn automatically by experimenting and implementing different tactics.

In warehouse management, RL provides powerful tools to improve the various aspects of the operation such as inventory prediction, restocking, selecting the best warehouse location among all. By data-driven decision-making, RL can make the operation more efficient, reduce costs, and improve overall operational performance in supply chains, and making it well-suited for complex and dynamic environments to solve the problems of inventory management, warehouse allocation, and best warehouse selection among all.

Deep Q-Network (DQN): The Deep Q-Network (DQN) is an optimized extension of traditional Q-learning that utilizes deep neural networks to approximate the Q-value function, making it scalable to problems with large and continuous state-action spaces. Instead of storing and updating a tabular Q-table, the DQN learns a function that maps state-action pairs to expected rewards through supervised learning, enabling it to generalize across high-dimensional environments. As demonstrated in the PyTorch DQN tutorial [8], techniques such as experience replay and target networks are

employed to stabilize training and improve convergence. In the context of warehouse management, DQN offers significant advantages by efficiently predicting optimal inventory replenishment actions, even when faced with large, dynamic datasets representing inventory levels, lead times, and fluctuating demand patterns.

Proximal Policy Optimization (PPO): Proximal Policy Optimization (PPO) is a policy-gradient reinforcement learning algorithm that has proven effective for solving complex decision-making tasks, particularly those involving continuous action spaces. Unlike value-based methods like DQN, PPO directly optimizes the policy by balancing exploration and exploitation while maintaining stable and reliable training. As explained in the PyTorch PPO tutorial [9], PPO uses a clipped surrogate objective function that prevents large, destabilizing updates to the policy network, ensuring more gradual and consistent learning. This makes PPO particularly suitable for high-variance environments such as warehouse management, where continuous restocking decisions and dynamic supplier selections are necessary. In this project, PPO facilitates adaptive decision-making under uncertainty, optimizing warehouse location selection in a way that traditional discrete-action models may struggle to handle.

RL provides a powerful framework for optimizing warehouse operations by enabling autonomous decision-making, reducing human intervention, and adapting to changing conditions in real-time.

2.3.1 Deep Q-Networks (DQN) for Inventory Forecasting

Deep Q-Networks (DQNs) are an advanced class of value-based reinforcement learning (RL) algorithms that successfully combine the best aspects of Q-learning and deep learning. In standard Q-learning, the agent maintains a Q-table that estimates the expected cumulative reward for taking a specific action in a particular state. However, this approach becomes impractical in large or continuous state-action spaces due to the exponential increase in dimensionality. Therefore, DQN replaces the static Q-table with a deep neural network that approximates the Q-value function, allowing the agent to generalize across similar states and actions.

In the domain of warehouse management, DQNs have emerged as a powerful tool for inventory prediction and restocking strategies. Warehouses typically operate in dynamic environments characterized by fluctuating customer demands, variable supplier lead times, and space constraints. As discussed by Ilaria Giannoccaro et al. [10] and further explored by Tao Zhou et al. [11], DQN-based systems enable autonomous agents to learn optimal replenishment policies through self-interaction in simulated or real environments. By engaging in trial-and-error learning, the agent continuously updates its strategy based on observed rewards, gradually adapting its choices to maximize long-term performance.

A DQN-based inventory forecasting system typically processes input features such as current inventory levels, historical sales patterns, item pricing, and available storage capacity. The output action recommends the optimal quantity to reorder for each item at any given decision point. Over several training episodes, the agent learns to recognize complex patterns in demand data and adjusts its inventory ordering behaviour accordingly. This adaptability makes DQN particularly advantageous in high-volatility environments, where traditional static models like Economic Order Quantity (EOQ) or Reorder Point (ROP) often fail to maintain service levels or cost efficiency.

2.3.2 Proximal Policy Optimization (PPO) for Restocking Location Selection

Proximal Policy Optimization (PPO) is an advanced reinforcement learning algorithm belonging to the policy-gradient family, designed to address the instability, inefficiency, and sensitivity to hyperparameters that often plague traditional policy-gradient methods. As introduced by Schulman et al. [12], PPO improves stability during training by using a clipped surrogate objective function, which restricts the extent to which the updated policy can deviate from the old policy at each step. This constraint ensures a more balanced trade-off between exploration and stability, enabling more reliable convergence, particularly in environments with continuous and high-dimensional action spaces.

In the context of warehouse management, PPO offers distinct advantages over value-based methods like Q-learning or Deep Q-Networks (DQNs), which estimate action-values indirectly and may suffer from unstable policy improvements in complex environments.

Unlike DQNs, PPO directly learns a mapping from states to actions, enabling it to operate more effectively in settings where decisions involve continuous variables, such as prioritizing suppliers based on regional demand and operational constraints. As detailed by Engstrom et al. [13], careful attention to PPO's implementation details, such as appropriate choice of clipping parameters and batch sizes, further enhances its robustness and practical deployment in real-world applications.

Moreover, PPO can be applied to **restocking location prediction**, wherein the agent must determine the optimal warehouse or distribution center to fulfill inventory restocking needs for different regions. This problem involves complex considerations such as transportation costs, regional demand variability, lead times, and current warehouse capacity — a high-dimensional, dynamic decision space that PPO handles effectively. For instance, in a large distribution network, a well-trained PPO agent can not only predict which warehouse should **fulfil** an order but also determine the precise amount of inventory that should be moved from each source location to balance stock levels across the network. Such intelligent decision-making greatly enhances supply chain efficiency by reducing transportation costs, minimizing stockouts, and ensuring timely order **fulfilment**, which are critical objectives in modern logistics systems.

Reinforcement Learning Model	Key Characteristics	Applications
Deep Q-Network (DQN)	Combines Q-learning with deep learning; approximates Q-value function.	Inventory forecasting, optimal ordering policies in dynamic environments.
Proximal Policy Optimization (PPO)	Policy-gradient method with clipped objective function for stability.	Restocking location selection, dynamic supplier selection.
Q-learning	Value-based approach where agents learn optimal actions by maximizing cumulative rewards.	Basic inventory control, simple decision-making problems.
Actor-Critic	Uses both policy and value functions to learn optimal strategies.	Multi-agent systems, warehouse allocation decisions.

Table 2.3 (a) Reinforcement Learning Techniques

Technique Type	Accuracy	Adaptability	Decision-making Autonomy	Computational Complexity
Traditional Techniques	Moderate to Low	Low (static assumptions)	Low (rule-based)	Low
Machine Learning Models	High (complex patterns, seasonal trends)	Moderate (adaptable to trends)	Moderate (prediction-based)	Moderate to High (requires data)
Reinforcement Learning Models	Very High (optimizing real-time decisions)	High (learning through interaction)	High (autonomous, dynamic)	High (requires training and simulation)

Table 2.3 (b) Comparison of Techniques

Chapter 2 presents a literature review of the development of inventory management from rule-based traditional models to machine learning and reinforcement learning-driven intelligent systems. Although traditional models provide simplicity, they are not adaptable and responsive. Machine learning enhances forecasting accuracy but is constrained in autonomous decision-making. Reinforcement learning presents itself as a viable solution that learns optimal restocking and supplier policies, especially in dynamic and distributed warehouse environments.

Machine learning integrated into warehouse management systems holds potential solutions for most of the problems of the traditional methods. DQN and PPO are two efficient reinforcement learning algorithms which can be utilized to optimize the decision of predicting and select the best warehouse from where to bought the items. The integration of a real dataset, multi-warehouse consideration, and autonomous functionality points the project's contribution to addressing existing gaps in literature.

Commented [RK4]: Mention in brief about what is there in the next chapter.

CHAPTER 3

DATA PREPROCESSING

In reinforcement learning (RL)-based systems, data preprocessing plays a different role than in traditional machine learning. Instead of cleaning large datasets, preprocessing in RL is aimed at preparing **state representations** for the agent such that environmental observations are structured, normalized, and interpretable.

The environment state consists of multiple features including inventory levels, warehouse locations, demand forecasts, and agent-specific information like action history or step count. These features need to be pre-processed into a compact, numerical form before they can be used for learning and to transform raw inventory and warehouse metadata into structured formats compatible with the learning environments, ensuring robustness, reliability, and performance during training and prediction.

3.1 Raw Data Overview

3.1.1 Inventory Data (dateItems)

The inventory data is stored in MongoDB under each warehouse document. It is structured by date and contains the inventory status of individual items, including:

```

  ▾ 2: Object
    date : 2025-04-11T00:00:00.000+00:00
    ▾ items : Array (2)
      ▾ 0: Object
        id : "32"
        inventory : 47
        sale : 0
        name : "dsgv"
        price : 45
        space : 47
      ▾ 1: Object
        id : "944211"
        inventory : 42
        sale : 0
        name : "ehr"
        price : 23
        space : 42

```

Fig 3.1 (a) Raw data of DateItems

Commented [RK5]: Draw a table

3.1.2 Warehouse Metadata

Each warehouse also contains information about location, name, and optionally trained models and prediction history.

```

_id: ObjectId('67f628a82b94312b5fe8d1e1')
name: "ergf"
location: "Umred Road, Reshimbag, Gandhi Baug, Nagpur, Nagpur Urban Taluka, Nagpu..."
▸ coordinates: Object
  lat: 21.148392478869828
  lon: 79.89986568496693
▸ dateItems: Array (3)
▸ trainedModel: Object
  32: Binary.createFromBase64('gASVsn8AAAAAACMC2NvbGx1Y3Rpb25zLiwLT3JkZXJlZERpY3SUk5QpUpQojAtmYy4wLndLaWdodJSMDHRvcml9dG1sc5SML', 0)
  944211: Binary.createFromBase64('gASVsn8AAAAAACMC2NvbGx1Y3Rpb25zLiwLT3JkZXJlZERpY3SUk5QpUpQojAtmYy4wLndLaWdodJSMDHRvcml9dG1sc5SML', 0)
userId: ObjectId('67f6285e2b94312b5fe8d1db')
__v: 4

```

Fig 3.1 (b) Raw Metadata of Warehouse

3.2 Inventory Forecasting (DQN) Preprocessing

The DQN agent is trained to learn an optimal restocking policy based on historical item-wise inventory data. Preprocessing steps include:

3.2.1 Flattening Nested Records

Each day's record for all items is flattened into individual entries with the following structure:

plaintext

```
[timestamp, item_id, inventory, sale, price, space]
```

- Timestamp is parsed from ISO format to UNIX timestamp.
- Only records with both sale and inventory present are taken into account.
- If fields like price or space are missing, they are automatically set to 0.

Format:

```
[  
  [1706745600.0, 'item1', 50, 10, 100, 100],  
  [1706832000.0, 'item2', 40, 5, 100, 100],  
]
```

3.3 Restock Location Selection (PPO) Preprocessing

The PPO agent determines the best warehouse for restocking by evaluating multiple factors. The following steps are used to preprocess this data:

3.3.1 Input Format

A JSON array of candidate warehouses is supplied via stdin. Each warehouse entry includes:

- warehouseName
- distanceKm: Distance from the central node
- durationMin: Time taken to reach the warehouse
- items: Array of items with restockedAmount

A new feature restockScore is computed as the total number of items restocked at that location:

python

```
[restockScore, distanceKm, durationMin]
```

This is used as the observation space for the PPO agent, which evaluates and selects the warehouse that maximizes restock benefit while minimizing cost and delay.

3.4 Handling Missing and Corrupt Data

- Inventory entries with missing inventory or sale fields are skipped.
- Default values are used for optional fields (price, space) to prevent training crashes.
- Error handling during datetime parsing ensures model robustness:

Commented [RK6]: At the end of every chapter write a brief summary of the chapter and what is to be expected in the next chapter.

```
function formatDateToISO(date) {  
    const d = new Date(date); // Convert the date string to a Date object  
    return d.toISOString().split('T')[0]; // Extract YYYY-MM-DD  
}
```

Fig 3.4 (c) format date

CHAPTER 4

Commented [RK7]: Title should be just after this.

In this chapter we will discuss the design of models implemented, libraries and performance metrics used, and the implementation.

DESIGN AND IMPLEMENTATION

In this chapter the design and implementation of the machine learning models used in the warehouse management system. Deep Q-Network (DQN) model is used for inventory prediction and the Proximal Policy Optimization (PPO) model is used for the best warehouse location selection for all the warehouses. The **models' code is** implemented in Python, with the backend system powered by Node.js using Python scripts to handle the computational tasks. The backend interacts with the frontend via REST APIs and integrates the machine learning is run in the backend and it provide the information which is needed to display.

4.1 System Components

The smart warehouse management system is divided into several key components, each of which plays a critical role in the overall functionality of the system. These components include:

- **Frontend:** The user interface with which users will interact with the website. Users can use it to display inventory data, enable inventory prediction and optimum warehouse location selection functions.
- **Backend:** The program operating on the server, and facilitating communication between the frontend and the machine learning models. It is developed with Node.js and Express.js.
- **Machine Learning Models:** The fundamental decision-making processes are handled by the reinforcement learning models (DQN for inventory prediction and PPO for restocking optimization).
- **Database:** To store warehouse inventory data, warehouse name, location, and other details, we have used a MongoDB database.

4.1.1 Libraries

Module	Description
User Authentication	Login, registration
Warehouse Management	Add/view warehouses and item details per day
Inventory Forecasting	Uses DQN model for predicting restocking needs
Location Optimization	Uses PPO for assigning restock locations dynamically
Frontend	Interactive UI with Leaflet-based warehouse map integration

Table 4.1 (a) – Libraries and Frameworks Used

NumPy – NumPy [14] is the "Numerical Python" library supporting large, multi-dimensional arrays and matrices. It is applied to different computations like linear algebra, Fourier transformations, and statistics. NumPy is the core library for scientific computing with Python.

PyTorch – PyTorch [15] is a Facebook AI Research Lab-developed open-source machine learning library. PyTorch is a library for building and training neural networks with dynamic computation graphs. PyTorch is utilized in this case to execute deep reinforcement learning models, both DQN and PPO agents.

Gym – Gym [16] is an OpenAI-developed toolkit for constructing and contrasting reinforcement learning algorithms. While there was no default environment, Gym provided a base on which a bespoke warehouse environment was constructed. Gym offers a unified API to simulate environments and provides support for interacting between the environment and agent through `step()` and `reset()` methods.

PyMongo – PyMongo [17] is the official Python driver for MongoDB which is a NoSQL database. It makes it possible for Python programs to conduct queries, save documents, and read and write MongoDB collections. PyMongo is used in this project to store and load model training results, warehouse data, and future inventory predictions.

BSON – SON (Binary JSON) [18] is a binary-encoded serialization format used to store documents in the bson module's ObjectId and Binary types are used to serialize trained model weights before they are stored in MongoDB and to maintain unique identifiers for warehouse entries.

Pickle – Pickle [19] is a module it is used to serialize and deserialize Python objects. In our project we have loading the trained PyTorch model weights again for prediction after converting them into binary format for MongoDB storage.

Datetime – Datetime [20] module provides classes for working with dates and times. For model input and MongoDB storage, it is utilized to parse historical inventory timestamps and convert between various date formats.

Collections – Python's built-in collections [21] module provides specialized container data types. The deque class is used to implement experience replay buffers in DQN, enabling the storage and sampling of past experiences to improve learning stability and reduce correlation between training samples.

Random – The random module [22] is part of Python's standard library and is used to add stochasticity to action selection and environment simulation during the DQN exploration stage. It has functions for generating random numbers.

4.2 Frontend Architecture

Frontend of the project is a web interface that is responsive and which is meant to display warehouse inventories, restocking options, and the best selection of warehouses from among the all. Users can interact with the reinforcement models and track the performance of the system.

The frontend is designed using HTML, CSS, and JavaScript. It provides interactive dashboards and interfaces for warehouse managers. Key features include:

The Frontpage when logged in show a brief summary of the warehouses.

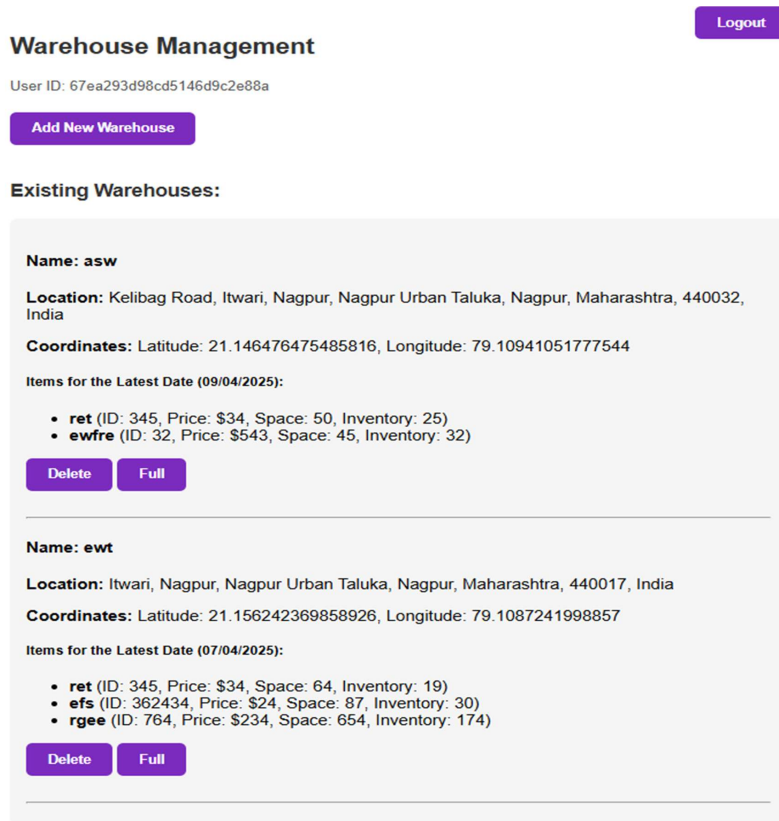


Fig 4.2 (a) Frontend display when first loaded

When clicked on the “add new warehouse” button.

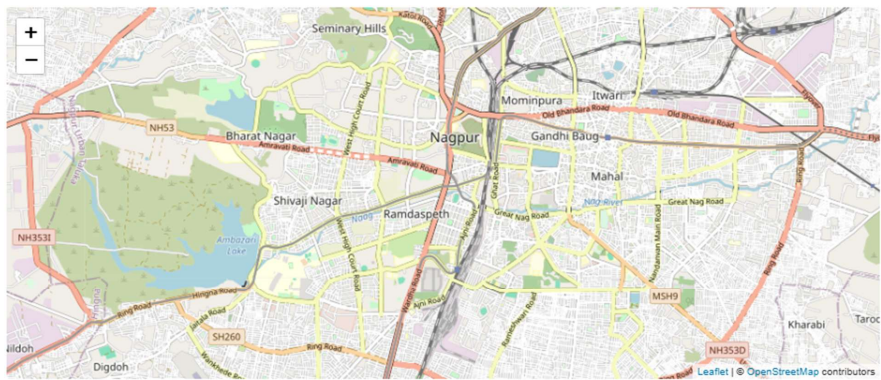
Warehouse Location & Items

Enter Warehouse Information

Warehouse Name:

Warehouse: Not Set

Search



Select Date:

Item ID:

Item Name:

Item Price:

Item max space allotted:

Item in Inventory:

Add Item

Items for Selected Date:

ID: 23 | Name: qw | Price: \$24 | Space: 45 | Inventory: 17 |

Save and Go to Main Page

Fig 4.2 (b) Frontend display when adding a new warehouse

Inventory Dashboard: Shows sales information, inventory levels, and available space for each product

Warehouse Details

Name: asw

Location: Kelibag Road, Itwari, Nagpur, Nagpur Urban Taluka, Nagpur, Maharashtra, 440032, India

Coordinates: Latitude: 21.146476475485816, Longitude: 79.10941051777544

Items by Date:

04-04-2025

Edit

Delete

Record Sales

- ret (ID: 345, Price: \$34, Space: 50, Inventory before sale: 39, sale: 34)
- ewfre (ID: 32, Price: \$543, Space: 45, Inventory before sale: 42, sale: 23)

05-04-2025

Edit

Delete

Record Sales

- ret (ID: 345, Price: \$34, Space: 50, Inventory before sale: 14, sale: 14)
- ewfre (ID: 32, Price: \$543, Space: 45, Inventory before sale: 45, sale: 4)

06-04-2025

Edit

Delete

Record Sales

- ret (ID: 345, Price: \$34, Space: 50, Inventory before sale: 11, sale: 11)
- ewfre (ID: 32, Price: \$543, Space: 45, Inventory before sale: 45, sale: 7)

07-04-2025

Edit

Delete

Record Sales

- ret (ID: 345, Price: \$34, Space: 50, Inventory before sale: 11, sale: 11)
- ewfre (ID: 32, Price: \$543, Space: 45, Inventory before sale: 45, sale: 43)

08-04-2025

Edit

Delete

Record Sales

- ret (ID: 345, Price: \$34, Space: 50, Inventory before sale: 28, sale: 4)
- ewfre (ID: 32, Price: \$543, Space: 45, Inventory before sale: 28, sale: 6)

Fig 4.2 (c) Inventory display

Map Visualization: Maps warehouse locations in real time using Leaflet and OpenStreetMap

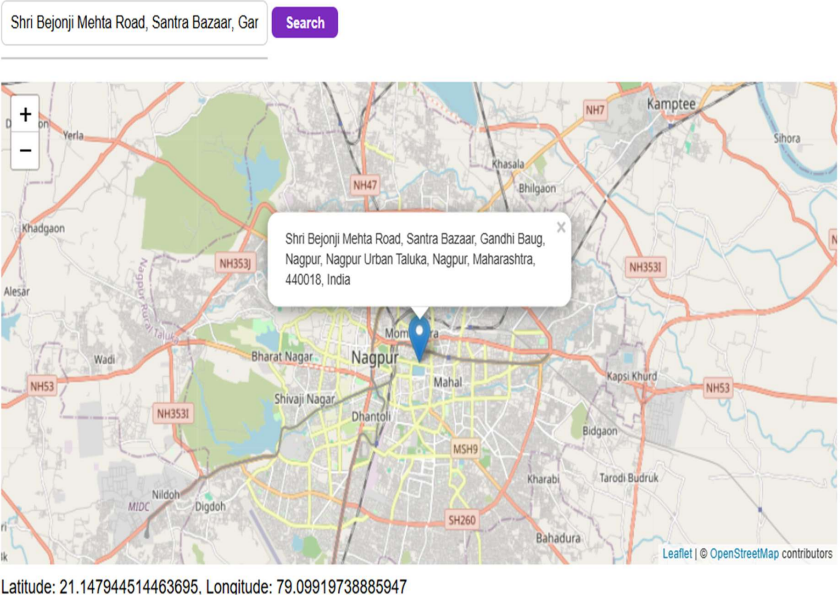


Fig 4.2 (d) Map Visualization

Forecasting Interface: Lets users to initiate inventory forecasting by sending data to the DQN model.

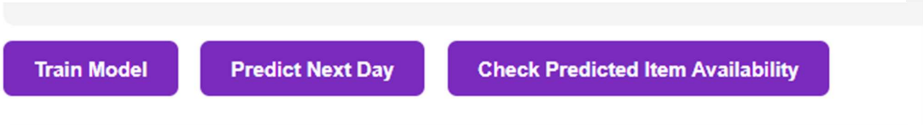


Fig 4.2 (e) Forecasting Interface

Restocking Interface: Displays the recommended restocking locations from the PPO model.

4.3 Backend Architecture

The backend receives requests handling and communicate with the machine learning models. It is built using Node.js and Express.js, which communicate with python scripts that handle the computational. Key components include:

- **API Routes:** Allowing communication between front and back end.
 - `/train/:warehouseId`: It start the training of the DQN model.
 - `/predict/:warehouseId`: It request inventory forecasts from the DQN model.
 - `/restock/:warehouseId`: It request the best warehouse to restock from the PPO model.
- **Machine Learning Integration:** The backend runs python scripts (such as `train_predict.py` and `ppo_location_selector.py`) using the child process module to handle the interactions.

4.4 Machine Learning Models

The system's foundation is its machine learning models, which give it the ability to make wise decisions. In this system, two models are used:

- DQN for Inventory Forecasting
- PPO for Warehouse Location Selection

4.4.1 DQN for Inventory Forecasting

The Deep Q-Network (DQN) model is a reinforcement learning model that employs deep neural networks to estimate Q-values. The Q-values are the future rewards of actions taken in a state. The states are the current inventory levels, and the actions are the decisions like how much the number of items to order for the inventory prediction. The objective of the DQN agent is to learn the optimal replenishment policy by acting within the environment. By the current ongoing exploration and feedback, the DQN agent reduces the overall cost of overstocking and stockouts. In the long run, it learns to place proactive orders that accommodate changing demand and finite storage space.

State and Action Space

- **State Space:** The state space includes a vector that represents the current state of the warehouse, with features including
 - current inventory levels
 - Sales forecasts
 - item prices
 - space available
- **Action Space:** The action space consists of discrete actions like ordering a certain number of items (i.e. 10, 20, or 30 units).
 - Restocking values range from 0 to the maximum allowable available space in the warehouse. For example, if a warehouse has maximum space for 27 more units, then the action space would be:
 - $A = \{0, 1, 2, \dots, 27\}$

Model Architecture

The DQN model is built with a neural network with the following architecture:

- **Input Layer:** A dense layer taking the state vector (inventory levels, sales, price, space, id that is length = 5).
- **Hidden Layers:** Two fully connected layers with 64 neurons and ReLU activation functions for learning complex relationships between states and actions.
- **Output Layer:** A dense layer with linear activation to output Q-values for each action.

It is trained with Mean Squared Error (MSE) as the loss function and Adam optimizer to reduce the discrepancy between expected and true Q-values.

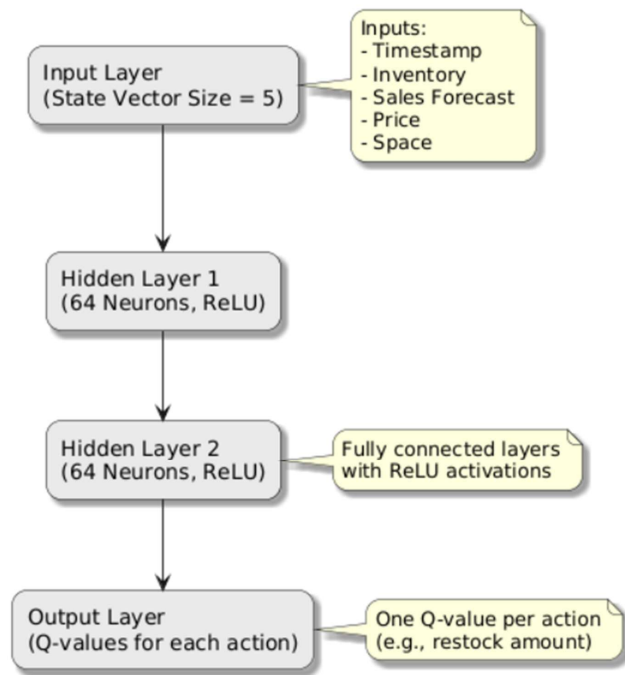


Fig 4.4 (a) DQN model Architecture

Training the Model

The backend triggers the training of the DQN models when new data is available and the user choose to train the model from frontend. To make sure the models are adjusting to new trends and pattern, they are periodically updated after being trained on historical data. In the training process the agent chooses actions, observes rewards, interacts with the environment, and updates its Q-values. This training steps tell how the training loop work:

- **Initialization:** The agent is given the starting state after the environment has been reset.
- **Action Selection:** On the current state the agent selects an action. This is done using an epsilon-greedy policy, where the agent sometimes explores random actions and other times exploits its learned knowledge.

- **Environment Step:** The next state and related reward are returned after the action is applied to the environment.
- **Q-Value Update:** The Q-values of the model is updated by using the Bellman equation to incorporate the observed reward and the predicted future reward.
- **Repeat:** The process is repeated for a specified number of episodes, during which the agent learns the optimal strategy.

The model has been trained, saved to disk, and is available there for predictions in the future.

Reward Function Design

The reward function is designed to balancing the stock availability:

- **Fulfilment Reward:**
 - Positive reward for keeping leftover inventory between 5–25% of total space.
 - Heavy penalty for over stocking or under stocking the inventory.
- **Restock Penalty:**
 - The restock penalty is calculated based on the difference between the ideal restock quantity (sales forecast) and the actual restock action taken by the agent. The penalty is proportional to the difference, so that it encourages the agent to make precise decision
 - Penalizes deviation from actual sales to encourage precise restocking.

$$\text{RestockLoss} = \lambda \cdot |\text{RestockQty} - \text{ActualSales}|$$

- Where $\lambda=0.2$ is the penalty coefficient.
- **Reward Calculation:**
 - The total reward for each state-action pair is calculated as follows:

$$\text{Reward} = \text{Fulfillment Reward} - \text{Restock Penalty}$$

where the fulfilment reward depends on the leftover inventory ratio, and the restock penalty is subtracted from the reward.

Predicting Inventory Levels

After the model is trained, it is used to predict the inventory quantity for the next day how much to restock. The prediction follows these steps:

- Fetch the latest state from MongoDB (dateItems collection).
- Construct the state vector for each item.
- Load the trained DQN model from the database.
- Pass the state through the model to get the action (i.e., restock amount).
- For the next day the MongoDB is updated with the predicted inventory.

Code Implementation

- **dqn_agent.py**: Contains the agent class, DQN model, replay memory, and target network handling.
- **warehouse_env.py**: Defines the environment, state transitions, and reward logic.
- **train_predict.py**: Orchestrates training using the environment and agent classes, and saves models.

```
class DQN(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(DQN, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 64),
            nn.ReLU(),
            nn.Linear(64, output_dim)
        )

    def forward(self, x):
        return self.fc(x)
```

Fig 4.4 (b) DQN model Architecture Code

4.4.2 PPO for Warehouse Location Selection

Proximal Policy Optimization (PPO) is a reinforcement learning algorithm which is designed to handle continuous action spaces and high-dimensional environments. In our project of warehouse management, the PPO model is used to recommend best warehouse locations among all the warehouse for item restocking based on demand patterns, delivery time, and distance.

State and Action Space

- **State Space:** Each state is a vector consisting of:
 - restockScore: Addition of all the quantity of restock.
 - distanceKm: The warehouse from which we are ordering distance between it.
 - durationMin: Estimated delivery time to reach the warehouse.
- **Action Space:** A discrete set of potential warehouses where an item can be stored or restocked.

Model Architecture

The PPO model is built with a neural network with the following architecture:

- **Input Layer:** A dense layer taking the state vector (restockScore, distanceKm, durationMin).
- **Hidden Layers:** Fully connected layers with ReLU activation functions for learning complex relationships between states and actions both the actor and critic share the hidden layer.
- **Actor Network:** The action probabilities are output by the actor, which tell about the possible storage locations of the item. Using SoftMax function the Action probabilities are calculated. In order to enable the agent to choose the most appropriate location according to the current state.
- **Critic Network:** The current state value is estimates by critic. This predicted value represents how good or bad the current state is, helping the model understand the expected long-term reward of selecting certain storage locations.

The model uses **Adam optimizer** with a learning rate of 0.002 using a policy gradient method.

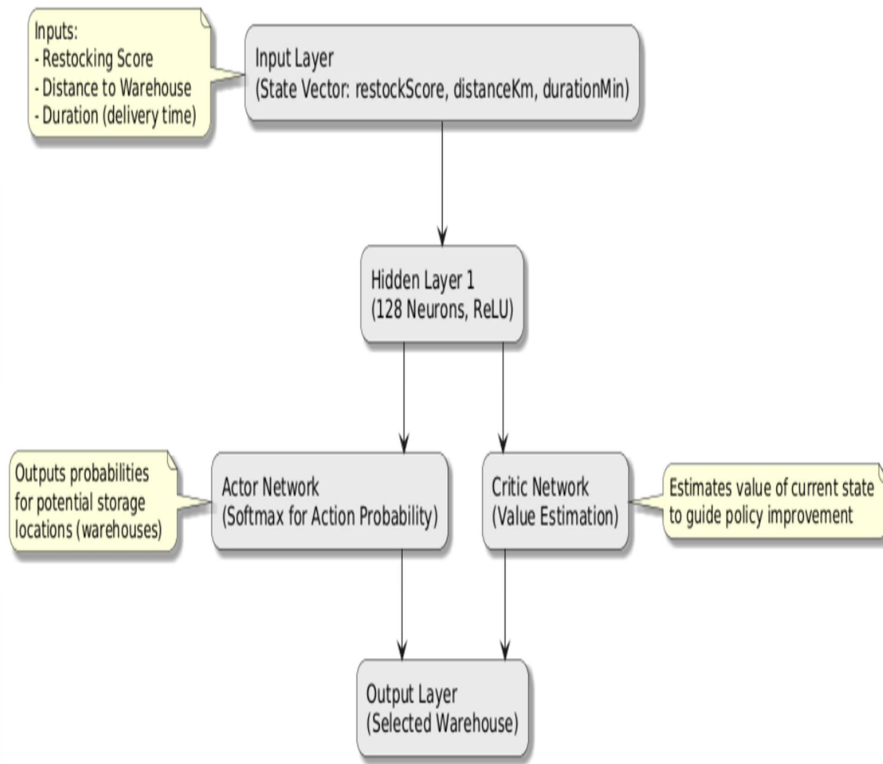


Fig 4.4 (c) PPO model Architecture

Training the Model

The training of the PPO model involves interactions with the warehouse environment. The agent selects storage locations based on the current state in which the inventory is there which was predicted by the model, and it receives rewards based on the efficiency of the chosen locations. By ensuring effective policy updates and clipping the objective function, the PPO algorithm stabilizes the training process.

The PPO training loop consists of the following steps:

- **Action Selection:** Based on the current state the agent selects an action (storage location) using a probability distribution generated by the actor network. The action is selected using **SoftMax** distribution of action probabilities.
- **Reward Calculation:** After selecting an action, the agent receives a reward based on how efficient the selected location is in terms of retrieval time, Distance in Km, and item demand.
- **Policy Update:** Using the surrogate objective function the policy is updated where we try to maximize the expected reward while ensuring that updates which we have done to the policy do not deviate too far from the old policy. Using the clipped objective function is used to maintain the stability.
- **Critic Update:** The critic network is updated by minimizing the loss between the predicted value and the actual return.
- **Repeated Iterations:** The process is repeated for multiple episodes, where the model learns to optimize the storage locations over time by refining its policy.

Reward Strategy

$$\text{Reward} = 10 \times \text{restockScore} - 0.5 \times (\text{distanceKm} + \text{durationMin})$$

The agent's reward function is designed to balance two important objectives:

- **High utility:** Prioritize storage locations with higher demand (restockScore).
- **Low delivery cost:** Minimize the distance and delivery time associated with each warehouse.

The reward function combines these factors by rewarding higher restock scores and penalizing higher distances and durations. This ensures that the agent learns to prioritize warehouses that are both cost-effective and meet the demand requirements.

Predicting Optimal Storage Locations

Once the PPO model is trained, it can be used to predict the optimal storage location for each item in the warehouse. The model selects the storage location that maximizes the

expected reward, based on the current state of the warehouse. This process is repeated for each item, ensuring efficient space utilization and minimizing retrieval time.

The prediction process involves the following steps:

- **State Input:** The current state of the warehouse (restockScore, distanceKm, and durationMin) is fed into the trained PPO model.
- **Action Selection:** The agent selects the optimal warehouse based on the current state.
- **Output:** The selected warehouse is returned as the predicted optimal storage location.

Code Implementation

The PPO model is implemented in the ppo_location_selector.py file. It includes the actor-critic architecture, action selection, reward computation, and policy updates. The training loop interacts with the warehouse environment, and the model is trained to optimize warehouse location decision. And it consists of the neural network architecture and training functions.

```
class ActorCritic(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(ActorCritic, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU()
        )
        self.actor = nn.Linear(128, output_dim)
        self.critic = nn.Linear(128, 1)

    def forward(self, x):
        x = self.fc(x)
        return self.actor(x), self.critic(x)
```

Fig 4.4 (d) PPO model Architecture Code

4.5 Database Architecture

Commented [RK8]: Should come early in the chapter

The system is built using MongoDB database. The system will use MongoDB to store warehouse data, sales records, and information regarding our machine-learning models. Similar to relational database systems, it is traditional in NoSQL-based systems to use some type of schema or rules for its construction. Below are the collections in our schema:

Users: Stores registered users along with authentication credentials and associated warehouses.

Warehouses: Stores warehouse information, location, daily inventory records, and associated ML models.

4.5.1 User Collection

- This collection will hold an identification number, username, email address, and hashed password about each user.
- The identification numbers by whom the warehouses are owned and operated by each user; this is important for linking users and warehouses.
- It ensures email uniqueness and security through lowercase enforcement and password hashing for authentication.

Field	Type	Description
_id	ObjectId	Unique identifier for the user
username	String	Username for login
email	String	Email address (unique, lowercase)
password	String	Hashed password
warehousesId	ObjectId []	List of warehouse IDs owned by the user

Table 4.5 (a): Users Collection Schema


```

_id: ObjectId('67fb589253af8a46b246c08a')
username: "Z"
email: "z@z.z"
password: "$2a$10$I3W7WL4qedjtPYRXvbWWF03G9cgRPNCTfDHsoIVugn2ZF8p0vhg0u"
warehousesId: Array (2)
__v: 7

```

Fig 4.5 (b) User Database structure

Relationships:

- One-to-Many: A user can own multiple warehouses.

4.5.2 Warehouse Collection

- Stores information about each warehouse, including its ID, name, location, coordinates, date items, inventory, trained models.
- Stores information about the particular date and on that date items in each warehouse and its id, name, price, space, inventory and sales.
- Trained models for forecasting (stored as binary or serialized object)
- Stores the trained machine learning models using pickle.

Field	Type	Description
name	String	Name of the warehouse
location	String	Physical location address
coordinates	Object	Latitude and longitude
dateItems[]	Array	Daily grouped items with inventory & sales
trainedModel	Object	Stored ML model (binary) per item ID
userId	ObjectId (ref)	Links to user who owns the warehouse

Table 4.5 (c): Warehouse Collection Schema

```

_id: ObjectId('67f628a02b94312b5fe8d1e1')
name: "ergf"
location: "Umred Road, Reshimbag, Gandhi Baug, Nagpur, Nagpur Urban Taluka, Nagpu..."
coordinates: Object
  lat: 21.140392478869828
  lon: 79.09986568496693
dateItems: Array (3)
trainedModel: Object
  32: Binary.createFromBase64('gASVsn8AAAAAACMC2NvbGx1Y3Rpb25zLTlWLTJkZkZlZERpY3SUK5QpUpQojAtmVy4wLndlaWdodJSMdHRvcml91dG1sc5SM_', 0)
  944211: Binary.createFromBase64('gASVsn8AAAAAACMC2NvbGx1Y3Rpb25zLTlWLTJkZkZlZERpY3SUK5QpUpQojAtmVy4wLndlaWdodJSMdHRvcml91dG1sc5SM_', 0)
userId: ObjectId('67f6285e2b94312b5fe8d1db')
__v: 4

```

Fig 4.5 (d) Warehouse Database structure

4.6 Integration with the Backend and Frontend

The backend for the smart warehouse management system is developed in Node.js and coupled with the machine learning models by Python scripts. The backend is utilizing the `child_process.spawn` functionality to run the Python scripts dealing with forecasting the inventory (DQN) and choosing a location (PPO).

The frontend is developed with HTML, CSS, and JavaScript and serves as an interface for interacting with the system. Users can enter warehouse information and see real-time predictions for inventory levels and ideal storage locations.

The frontend communicates with the backend via **REST APIs**. If a forecast or restock location is asked by a warehouse manager, the frontend will pass the request on to the related API endpoint. The backend now executes the request, interacts with the database and retrieves the information required, calls the machine learning model (DQN or PPO), and the machine learning model takes data, makes the prediction, and sends back results to the backend. the backend sends them to the frontend, where they're presented in an end-user-friendly format. This process guarantees them to make optimum stock and restocking choices.

CHAPTER 5

Commented [RK9]: Title of the chapter ?

This chapter demonstrate the output of the system which we have implemented warehouse management system and the performance of reinforcement learning models DQN for inventory management and PPO to select the optimal warehouse location among all. The system was tested on realistic datasets and incorporated into a full-stack application consisting of a Node.js backend, MongoDB database, and a frontend using HTML/CSS/JS with interactive mapping.

RESULTS AND ANALYSIS

The system performance was evaluated based on the following factors:

- **Forecasting Accuracy:** How well the DQN model predicts inventory needs.
- **Restocking Decision Quality:** How well PPO choose the best restock locations based on distance, duration, and restock score.
- **Frontend-Backend Integration:** Easy interaction between user interface, data input, and real-time model output.
- **User Experience:** Warehouse managers' ability to use the web application.

5.1 Inventory Forecasting with DQN

5.1.1 Model Overview

The DQN model was trained for each item in each warehouse using the historical data that include timestamp, inventory, sales, price, and space. For each item:

- **State space:** 5-dimensional vector [timestamp, inventory, sale, price, space].
- **Action space:** Discrete restock amounts ranging from 0 up to the warehouse's space capacity.
- **Reward function:** Balanced fulfilment efficiency and overstock minimization using a reward strategy with penalties for restock deviation from actual demand.

5.1.2 Prediction Visualization

The frontend visually shows the displays of the next-day inventory predications. For example:

- Date: 2025-04-09
- Prediction Output:

```
{  
  "32": 32,  
  "345": 25  
}
```

These predictions are stored in MongoDB and then displayed on the dashboard, with past data also display with it to provide more information about it.

Prediction for 2025-04-09:

- ID 32: Inventory 32
- ID 345: Inventory 25

Fig 5.1 (a) Prediction result DQN

5.2 Warehouse Location Selection with PPO

5.2.1 Model Overview

The PPO agent was trained to choose the best possible warehouse among others based on:

- restockScore: Total number of restocked items, indicating utility.
- distanceKm: How close it is to the demand zone.
- durationMin: Delivery time.

State Vector: [restockScore, distanceKm, durationMin]

Action Space: Index of candidate warehouses.

5.2.2 Location Visualization

The best selected warehouse among the all is show on the top of the page.

Predicted Item Availability in Warehouses

Best Warehouse Recommendation
Name: ewt
Distance: 1.66 km
Time: 3 minutes
Warehouse: ewt
Distance: 1.66 km | **Estimated Time:** 3 min
• ret (ID: 345) - Price: 34, Inventory: 19, Restocked: 1

Fig 5.2 (a) Best warehouse PPO

5.3 Frontend Integration Results

5.3.1 Inventory Dashboard

The DQN model was integrated through `train_predict.py`, which executes through a spawn process in the Node.js backend. The frontend dashboard:

- Displays inventory levels and predicted stock.
- Accepts new item data via forms which is then stored in MongoDB and passed to the model.

Warehouse Details

Name: asw

Location: Kelibag Road, Itwari, Nagpur, Nagpur Urban Taluka, Nagpur, Maharashtra, 440032, India

Coordinates: Latitude: 21.146476475485816, Longitude: 79.10941051777544

Items by Date:

04-04-2025 [Edit](#) [Delete](#) [Record Sales](#)

- **ret** (ID: 345, Price: \$34, Space: 50, Inventory before sale: 39, sale: 34)
- **ewfre** (ID: 32, Price: \$543, Space: 45, Inventory before sale: 42, sale: 23)

05-04-2025 [Edit](#) [Delete](#) [Record Sales](#)

- **ret** (ID: 345, Price: \$34, Space: 50, Inventory before sale: 14, sale: 14)
- **ewfre** (ID: 32, Price: \$543, Space: 45, Inventory before sale: 45, sale: 4)

06-04-2025 [Edit](#) [Delete](#) [Record Sales](#)

- **ret** (ID: 345, Price: \$34, Space: 50, Inventory before sale: 11, sale: 11)
- **ewfre** (ID: 32, Price: \$543, Space: 45, Inventory before sale: 45, sale: 7)

07-04-2025 [Edit](#) [Delete](#) [Record Sales](#)

- **ret** (ID: 345, Price: \$34, Space: 50, Inventory before sale: 11, sale: 11)
- **ewfre** (ID: 32, Price: \$543, Space: 45, Inventory before sale: 45, sale: 43)

08-04-2025 [Edit](#) [Delete](#) [Record Sales](#)

- **ret** (ID: 345, Price: \$34, Space: 50, Inventory before sale: 28, sale: 4)
- **ewfre** (ID: 32, Price: \$543, Space: 45, Inventory before sale: 28, sale: 6)

09-04-2025 [Edit](#) [Delete](#) [Record Sales](#)

- **ret** (ID: 345, Price: \$34, Space: 50, Inventory before sale: 25, sale: undefined)
- **ewfre** (ID: 32, Price: \$543, Space: 45, Inventory before sale: 32, sale: undefined)

[Train Model](#)

[Predict Next Day](#)

[Check Predicted Item Availability](#)

Fig 5.3 (a) All days Item List after making prediction

5.3.2 Warehouse Location Map

Using **Leaflet.js** and **OpenStreetMap**, the PPO-based restocking system:

- Display all available warehouse nodes.
- Shows metrics such as delivery time, distance, and restock potential.

Predicted Item Availability in Warehouses

Best Warehouse Recommendation

Name: ewt

Distance: 1.66 km

Time: 3 minutes

Warehouse: ewt

Distance: 1.66 km | **Estimated Time:** 3 min

- ret (ID: 345) - Price: 34, Inventory: 19, Restocked: 1

Warehouse: ergf

Distance: 1.71 km | **Estimated Time:** 3 min

- dsgv (ID: 32) - Price: 45, Inventory: 47, Restocked: 10

Fig 5.3 (b) PPO all nearby warehouse were item available

From below images we can see the model daily sales, restock amount are performing in an optimal way

Commented [RK10]: Results can be shown in table format as well.

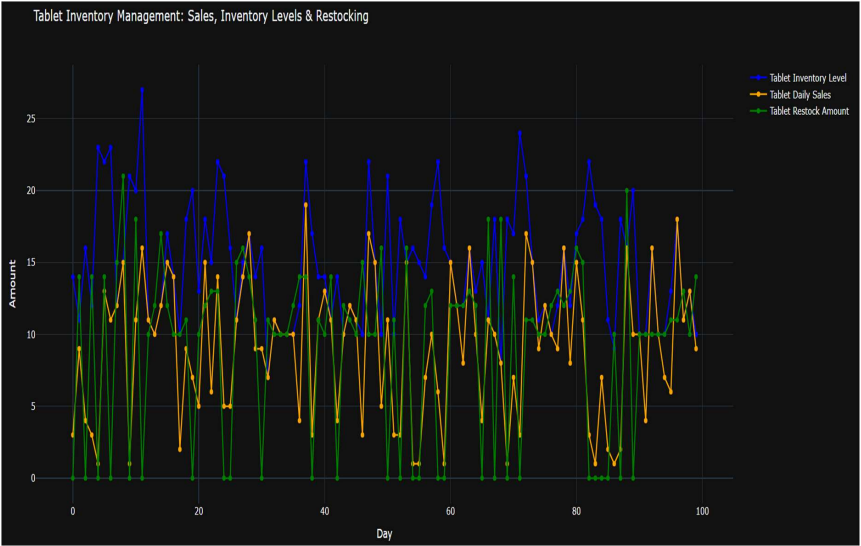
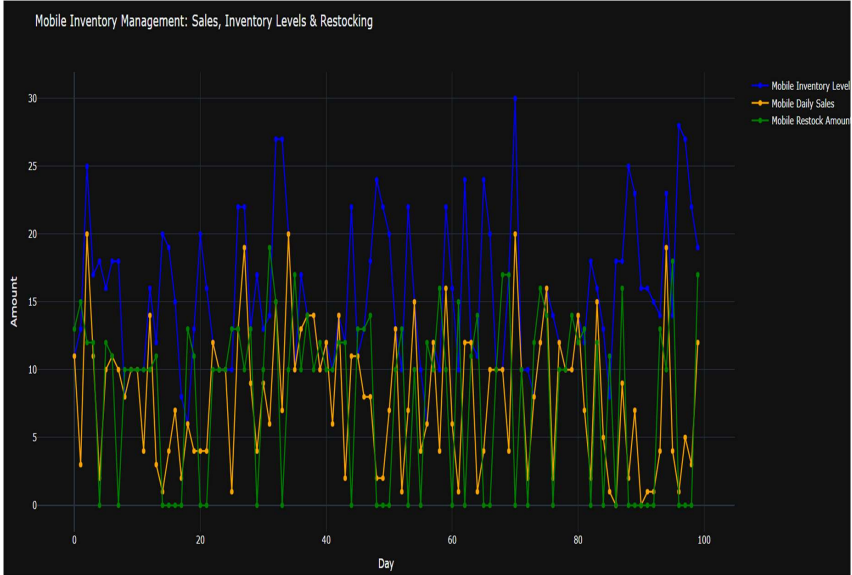


Fig 5.3 (c) Inventory of mobile and tablet graph

5.4 Comparative Analysis

Feature	DQN Inventory Forecasting	PPO Location Selector
Input Format	Time-series per item	Static feature vector per warehouse
Model Type	Value-based (Q-learning)	Policy Gradient (Actor-Critic)
Adaptivity	Learns over episodes	Learns over warehouse comparisons
Output	Optimal restock quantity	Optimal warehouse index

Table 5.4(a) DQN and PPO comparison

5.5 Summary

This chapter explain the results of both reinforcement learning models and their implementation into a working application. The system based on DQN gave accurate inventory predictions, while PPO allow the smart location choices. The frontend visualization tools helped bridge the gap between the user interaction with AI logic, making the resulting warehouse management improve and more usable.

CHAPTER 6

CONCLUSION AND FUTURE WORK

In this project, we designed and developed a full-stack **warehouse management system** that integrates machine learning algorithms with a practical web interface for efficient inventory control and location selection. We discover that supply chain management optimizations, where reward-based learning is very helpful, can be effectively demonstrated using reinforcement learning. The system uses **Deep Q-Network (DQN)** for dynamic inventory forecasting and **Proximal Policy Optimization (PPO)** to choose the best possible warehouse among others based on restock scores, distance, and travel time. Also, we discussed the 2 main techniques used in RL to solve one of the main issues of Supply chain that is inventory management and warehouse management.

- We used different techniques in Reinforcement Learning like Q-learning, DQN and PPO to improve the performance of the model shown on that website. Q-Learning, which provides a fundamental tabular approach to learning optimal policies in discrete environments.
- Deep Q-Networks (DQN), which extend Q-learning by using neural networks for estimating Q-values, thus making it scalable to large and continuous spaces.
- Proximal Policy Optimization (PPO), an advanced policy gradient method that ensures stable and effective learning by limiting drastic policy improvements.

The addition of these RL methods not only improved the performance of operations but also demonstrated the potential for change that AI can bring to the supply industry. As supply chains continuously becoming more and more complex and facing the challenges globally like demand uncertainty, transportation disruptions, and market unpredictability reinforcement learning stands out as a promising solution to all of this.

Future Work

While the current system which we have developed demonstrates promising results, several enhancements can be made in it:

1. Real-Time, Autonomous Supply Chains

- The current implementation uses batch-based simulation for training. Future work may focus on integrating real-time data pipelines to allow the system to continuously learn and adapt from live warehouse operations.

2. Integration with Digital Twins

- **Digital twins** (virtual representation of our physical systems) that can simulate supply chain environments where RL agents can be trained safely.
- RL agents interact with digital twins that enable testing of potential disruptions that can happen, fine-tuning of the processes, and prepare for rare but major events such as pandemics or natural disasters.

3. Improved Reward Design

- Reward functions can be further improved more by incorporating more business focused such as adding holding costs, stockout costs, or delivery deadlines to better reflect with real-world priorities that are there.

4. Advanced Frontend Features

The frontend interface can be further improved to include:

- Real-time alerts when the inventory is low.
- Interactive dashboards for ML model performance.

5. Sustainability and Green Logistics

RL can help reduce environmental impacts by:

- Optimizing fuel-efficient routes.
- Minimizing waste through better demand forecasting.

CHAPTER 7

REFERENCES

- [1] **Gayam, S.R.; Yellu, R.R.; Thuniki, P.** Optimizing Supply Chain Management through Artificial Intelligence: Techniques for Predictive Maintenance, Demand Forecasting, and Inventory Optimization. *Journal of AI-Assisted Scientific Discovery* 2021, 1(1). [Online]. Available: <https://scienceacadpress.com/index.php/jaasd/article/view/58>
- [2] **Sutton, R.S.; Barto, A.G.** Reinforcement Learning: An Introduction, 2nd ed.; MIT Press: Cambridge, MA, USA, 2018. [Online]. Available: <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>
- [3] **Mallick, R.K.; Patra, K.; Mondal, S.K.** A new economic order quantity model for deteriorated items under the joint effects of stock dependent demand and inflation. *Decision Analytics Journal* 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2772662223001285>
- [4] **Cheng, T.C.E.; Podolsky, S.** Just-in-Time Manufacturing: An Introduction; Springer Science & Business Media, 1996. [Online]. Available: https://books.google.co.in/books/about/Just_in_Time_Manufacturing.html?id=WL95vzpj1TIC&redir_esc=y
- [5] **Teuku Faisal Umry, Moses Laksono Singgih.** Inventory Management and Reorder Point (ROP) Strategy Using ABC Analysis Methods in Textile Manufacture. *IPTEK Journal of Proceedings Series*, 5, 2019. [Online]. Available: <https://www.smartmoses.com/wp-content/uploads/11-2019-Inventory-Management-and-Reorder-Point-ROP-Teuku-Faisal-Umry.pdf>

- [6] **Abbaspour Ghadim Bonab, A.** A Comparative Study of Demand Forecasting Based on Machine Learning Methods with Time Series Approach. *Journal of Applied Research on Industrial Engineering* 2022, 9(3), pp. 331–353. [Online]. Available: https://www.journal-aprie.com/article_127616_c9709b74e7808d865be111b8015d580d.pdf
- [7] **Cracan, C.** Retail Sales Forecasting Using LSTM and ARIMA-LSTM Hybrid Models. Bachelor Thesis, Erasmus University Rotterdam, 2020. [Online]. Available: <https://www.scribd.com/document/646981716/Cracan-Thesis>
- [8] **PyTorch.** Reinforcement Learning (DQN) Tutorial. [Online]. Available: https://pytorch.org/tutorials/intermediate/reinforcement_q_learning.html
- [9] **PyTorch.** Reinforcement Learning (PPO) Tutorial. [Online]. Available: https://pytorch.org/tutorials/intermediate/reinforcement_ppo.html
- [10] **Giannoccaro, I.; Pontrandolfo, P.** Inventory management in supply chains: A reinforcement learning approach. *International Journal of Production Economics* 2002, 78(2), pp. 153–161. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0925527300001560>
- [11] **Zhou, T.; Xie, L.; Zou, C.; Tian, Y.** Research on supply chain efficiency optimization algorithm based on reinforcement learning. *Advances in Continuous and Discrete Models* 2024, 2024(1). [Online]. Available: https://www.researchgate.net/publication/385596314_Research_on_supply_chain_efficiency_optimization_algorithm_based_on_reinforcement_learning
- [12] **Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O.** Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347, 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>

- [13] **Engstrom, K.; Ilyas, A.; Santurkar, S.; Tsipras, D.** The 37 Implementation Details of Proximal Policy Optimization. ICLR Blog Track, 2022. [Online]. Available: <https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/>
- [14] **NumPy.** NumPy Documentation. [Online]. Available: <https://numpy.org/doc/stable/>
- [15] **PyTorch.** PyTorch Documentation. [Online]. Available: <https://pytorch.org/docs/stable/index.html>
- [16] **OpenAI Gym.** Gym Documentation. [Online]. Available: <https://www.gymnasium.dev/>
- [17] **PyMongo.** MongoDB Driver for Python (PyMongo) Documentation. [Online]. Available: <https://pymongo.readthedocs.io/en/stable/>
- [18] **BSON.** BSON (Binary JSON) Documentation. [Online]. Available: <https://pymongo.readthedocs.io/en/stable/api/bson/index.html>
- [19] **Pickle.** Python Serialization Documentation (Pickle Module). [Online]. Available: <https://docs.python.org/3/library/pickle.html>
- [20] **Datetime.** Python Standard Library (Datetime Module). [Online]. Available: <https://docs.python.org/3/library/datetime.html>
- [21] **Collections.** Python Standard Library (Collections Module). [Online]. Available: <https://docs.python.org/3/library/collections.html>
- [22] **Random.** Python Standard Library (Random Module). [Online]. Available: <https://docs.python.org/3/library/random.html>